

Edge cases (search comes back empty) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `empty_search_practice.py` : an empty result list is the easy case — a real vector search almost never returns literally zero, it returns the closest chunks it has, relevant or not. **If not:** the Build Task's approval step would sometimes show a human a "grounded" answer that was actually built from thin, barely-relevant context.

Read both depths — they're not "wrong, right," they're 2 real, valid ways to solve the same problem, with real tradeoffs between them.

Basic Version

Approach 1 — the direct way

```
# empty_search_practice.py
def has_results(state):
    if not state.get("sources"):
        return "no_grounding"
    return "reason"

def cant_answer_node(state):
    return {
        "answer": "I don't have grounding for this in my documents.",
        "grounded": False,
    }

graph.add_node("cant_answer_node", cant_answer_node)
graph.add_conditional_edges(
    "search_node", has_results,
    {"no_grounding": "cant_answer_node", "reason": "reason_node"},
)

# test
result = graph.invoke({"task": "What's the weather like today?"})
print(result["answer"])
```

This works and correctly catches a completely empty result list. It doesn't catch the case of a thin, barely-relevant result coming back — only genuinely zero results.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — type hints, a `grounded` flag, and testing both paths

```
# empty_search_practice.py
def has_results(state: dict) -> str:
    sources = state.get("sources", [])
    if not sources:
        return "no_grounding"
    return "reason"

def cant_answer_node(state: dict) -> dict:
    return {
        "answer": "I don't have grounding for this in my documents.",
        "grounded": False,
    }

graph.add_node("cant_answer_node", cant_answer_node)
graph.add_conditional_edges(
    "search_node",
    has_results,
    {"no_grounding": "cant_answer_node", "reason": "reason_node"},
)

# test both paths
in_scope_task = "What does the document say about the vacation policy?"
in_scope = graph.invoke({"task": in_scope_task})
out_of_scope = graph.invoke({"task": "What's the weather like today?"})

assert in_scope["grounded"] is True
assert out_of_scope["grounded"] is False
print("Both paths behave correctly.")
```

Difference from Basic: full type hints. A `grounded: bool` field on every response, not just a message string — this lets any caller (the terminal UI, a test, a later API layer) check programmatically whether an answer is trustworthy, instead of parsing text to guess. And explicit assertions testing *both* the in-scope and out-of-scope paths in the same run, so a future change that breaks either path fails loudly and immediately. This version still only catches a genuinely empty list

— a search that returns a few barely-relevant chunks sails straight through to `reason_node`, which is what Approach 2 fixes.

Approach 2 — a relevance-score threshold, not just an empty-list check

Story: a thin, low-scoring result is just as misleading as no result at all — the list isn't empty, so the graph sails on to `reason_node` and the model dutifully "answers" from context that doesn't really support an answer. **If not:** the Build Task's approval step would show a human a confidently-worded answer with no way to tell it apart from one genuinely grounded in relevant documents.

```
# empty_search_practice.py
import logging

logger = logging.getLogger("grounding")
RELEVANCE_THRESHOLD = 0.75

def has_results(state: dict) -> str:
    scored_sources = state.get("scored_sources", [])

    if not scored_sources:
        logger.info("no_grounding reason=empty task=%r", state["task"])
        return "no_grounding"

    best_score = max(score for _, score in scored_sources)
    if best_score < RELEVANCE_THRESHOLD:
        logger.info(
            "no_grounding reason=below_threshold best_score=%.2f task=%r",
            best_score, state["task"],
        )
        return "no_grounding"

    return "reason"

def cant_answer_node(state: dict) -> dict:
    return {
        "answer": "I don't have grounding for this in my documents.",
        "grounded": False,
    }

graph.add_node("cant_answer_node", cant_answer_node)
graph.add_conditional_edges(
    "search_node",
    has_results,
```

```

    {"no_grounding": "cant_answer_node", "reason": "reason_node"},
)

```

`scored_sources` here is a list of `(chunk, score)` pairs — most vector retrievers can return the similarity score alongside each chunk if you ask for it (e.g. Chroma's `similarity_search_with_score`). A question with zero true overlap with the document set will often still get *some* chunk back, just with a low score — this catches that case, which the plain "is it empty" check cannot.

Approach 3 — same threshold check, plus a partial-grounding path

Story: sometimes search finds *something* relevant, but only a small slice of what the question is actually asking — a partial match, not a miss. Treating that identically to "no grounding at all" throws away something real; treating it identically to "fully grounded" risks answering the unmatched part anyway. **If not:** a real answer that quietly ignored half the question, because the context only covered half of it, would look identical to a fully-grounded one.

```

# empty_search_practice.py
PARTIAL_THRESHOLD = 0.55
RELEVANCE_THRESHOLD = 0.75

def has_results(state: dict) -> str:
    scored_sources = state.get("scored_sources", [])

    if not scored_sources:
        return "no_grounding"

    best_score = max(score for _, score in scored_sources)
    if best_score < PARTIAL_THRESHOLD:
        return "no_grounding"
    if best_score < RELEVANCE_THRESHOLD:
        return "partial_grounding"
    return "reason"

def partial_grounding_node(state: dict) -> dict:
    chunks = state["scored_sources"]
    context = "\n\n".join(chunk.page_content for chunk, _ in chunks)
    prompt = (
        "The following context may only partially answer the question. "
        "Answer only what the context actually supports, and explicitly "
        f"say what it doesn't cover.\n\nContext:\n{context}\n\n"
        f"Question: {state['task']}"
    )

```

```
draft = model.invoke(prompt)
return {"answer": draft.content, "grounded": "partial"}
```

`grounded` is no longer just `True / False` here — `"partial"` is a distinct, honest third state a caller can check for, instead of the graph being forced to round a partial match up to "trustworthy" or down to "nothing found."

Difference from Approach 1, and between Approaches 2/3: Approach 1's `has_results` only ever asks "is the list empty" — a real vector search rarely returns a truly empty list, it returns the closest chunks it has, relevant or not, so this check misses the more common failure mode. Approach 2 fixes that with a relevance-score threshold, plus logging every fallback so a real project can find patterns in what its document set is missing. Approach 3 goes one step further: instead of collapsing "somewhat relevant" into either "fully grounded" or "not grounded," it adds a third, honestly-labeled outcome for the middle ground.

Which one should you actually write? Approach 1's empty-list check is enough to ship first — it catches the obvious case for free. Move to Approach 2's threshold check once you've seen your retriever return low-relevance chunks for an out-of-scope question in testing (it will, on any real embedding model). Approach 3's partial-grounding path is worth adding once you've actually seen a real answer that quietly ignored part of the question because the context only covered part of it — a real, observed problem, not a hypothetical one, per this document's Core Concepts point about not answering from thin grounding.